二分典题

P1918 保龄球 - 洛谷

题解

(1) 题意 : 题目意思很简单 , 我输入一个数 , 能不能在数组里找到这个数并且返回下标

(2) 思路 : 题目的数据范围是 N , Q ≤100000 , 那么传统搜索 O(n) 肯定是不行了 , 那我们就需要使用一种更高效的算法: 二分查找 ! ! (时间复杂度O(log n))

(3) 需要注意的点 :

在使用二分搜素模板时 , 要注意左右指针都在给定范围的外面 ! 如果使用 a[mid] < x , 返回 a[r] ,求得的就是第一个等于 x 的数 ,如果使用 a[mid] ≤ x , 返回 a[l] , 则为最后一个等于 x 的数.

此外 , 二分搜索的数组必须满足单调性 , 因此我们需要对其进行排序 , 本题中我们使用了 pair 数据类型 , 其first我们存储数 , second 我们存储那个数的下标 , 排序默认是以first为基准 . 这样做的好处就是在排序后 , 我们也可以通过second知道这个数原本的下标.

代码(c++)

```
#include<bits/stdc++.h>

#define int long long

using namespace std ;
const int N = 1e5 + 10 ;

int n , t ;
pair<int , int> a[N] ;

void solve(){
    cin >> n ;
    for(int i = 1 ; i <= n ; i++){
        cin >> a[i].first ;
        a[i].second = i ;
    }

    sort(a + 1 , a + 1 + n) ;
    //默认以 a[i].first 为基准
    cin >> t ;
    while(t--){
        int l = 0 , r = n + 1 ;
        //保证 l 和 r 在数组边缘的外面
        int x ; cin >> x ;
        // x 为我们需要查找的数
        while(l + 1 < r){
            int mid = (l + r) / 2 ;
            if(a[mid].first < x){
                l = mid ;
            }
            else{
```

```
                r = mid ;
            }
        }
        if(a[r].first == x){
            cout << a[r].second << endl ;
        }
        else{
            cout << 0 << endl ;
        }
    }
}

signed main(){
    ios::sync_with_stdio(0) ;
    cin.tie(0) ,cout.tie(0) ;
    solve() ;
    return  0 ;
}
```

[P8647 蓝桥杯 2017 省 AB] 分巧克力 - 洛谷

题解

(1) 题意 : 我现在有 N 块巧克力 , 每一块都是一个长方形(大小为H[i] * W[i]) , 现在我想把这N块巧克力分给k个小朋友 , 这些巧克力必须一样大 , 并且得是正方形 , 请问想让这些巧克力尽可能大的话 , 分出去巧克力的边长是多少?

(2) 思路 : 我们可以在数轴上找到一个正方形对应的边长 , 这样的话那我们是不是需要找到符合题意的临界边长 , 此时,我们很自然地想到一个经典算法: 二分答案 ! ! !

代码(c++)

```
#include<bits/stdc++.h>
#define int long long

using namespace std ;

const int N = 1e5 + 10 ;
int n , k , h[N] , w[N] ;

bool check(int x){
    int sum = 0 ;
    for(int i = 1 ; i <= n ; i++){
        sum += (h[i] / x) * (w[i] / x) ;
        if(sum >= k) return true ;
        //采用剪枝思想 , 如果此时份数已经符合题意 , 那么接下去就不需要再check了
    }
    return (sum >= k) ;
}

void solve(){
    cin >> n >> k ;
    for(int i = 1 ; i <= n ; i++){
        cin >> h[i] >> w[i] ;
```

```
    }
    int l = 0 , r = 9999999999 ;
    while(l + 1 < r){
        int mid = (l + r) / 2 ;
        if(check(mid)){
            l = mid ;
            //sum >= k 说明分出去的巧克力≥题目要求的 , 此时我们想找的边长就 ≤ 答案
        }
        else{
            r = mid ;
        }
    }
    //我们找到了一个临界 l 左侧的部分就是 ≤ 答案的边长 , 右侧是>答案的边长 , 那么显然左侧最大的那个边长就是答案!

    cout << l << endl ;

    //二分答案红绿灯模板
}
signed main(){
    solve() ;
    return 0 ;
}
```

[P10909 蓝桥杯 2024 国 B] 立定跳远 - 洛谷

题解

(1) : 题意 : 我们现在有 n 个检查点 , 我们想找到一个最小的跳跃的长度 , 可以从起点跳到终点 , 我们可以多设置 m 个检查点 , 我们还可以用一次技能让自己跳跃的距离翻倍.

(2) : 思路 : 我们可以找到一个临界跳跃长度 , 使他正好能跳到终点 , 那么我们是不是需要从数轴上搜索这个值 , 因此我们很自然的想到了一个经典算法 : 二分答案 ! ! ! 遇到跳不过去的检查点 , 我们可以增加一个检查点 或者 使用一次技能 , 因此使用技能的本质和增加检查点是完全一样的 , 我们验证是否符合题意只需要用需要增加检查点的数量与m作比较就行了

代码(c++)

```
#include<bits/stdc++.h>

#define int long long

using namespace std ;

const int N = 1e5 + 10 ;
int n , m ;
int a[N] ;

bool check(int x){
    int cnt = 0 ;
    for(int i = 1 ; i <= n ; i++){
        cnt += (a[i] - a[i - 1] - 1) / x ;
        //如果间距和跳跃距离一样的话 , 会增加一次设置, 但实际上我们能跳过去 , 因此我们需要-1
    }
```

```
    return (cnt - 1  > m) ;
}

void solve(){
    cin >> n >> m ;
    for(int i = 1 ; i <= n ; i++){
        cin >> a[i] ;
    }
    int l = 0 , r = 99999999999 ;
    while(l + 1 < r){
        int mid = (l + r) / 2 ;
        if(check(mid)){
            l = mid ;
            //如果我们跳远的距离对应的增加数量>题目给定的数量 , 那么说明我们的跳跃距离太短
        }
        else{
            r = mid ;
        }
    }
    //我们找到了临界距离 , r对应的区间≥答案 , 因此我们想找最小的就直接输出r
    cout << r << endl ;
}
signed main(){
    ios::sync_with_stdio(0) ;
    cin.tie(0) , cout.tie(0) ;
    solve() ;
    return 0 ;
}
```